

Técnicas de Programación Concurrente I

Resumen completo — v3 (Distribuidos · Transacciones · Examen Tips)

Ing. Pablo A. Deymonnaz · FIUBA · Depto. de Computación

Incluye: Rust · Vectorización · Condvar · Problemas clásicos · Distribuidos · Transacciones · V/F Parcial

Unidad	Tema
0	Rust Esencial para Concurrency
1	Introducción — Definiciones y Modelos + tabla cuándo usar cada uno
2	Modelo Fork-Join y Vectorización
3	Programación Asíncrona
4	Corrección, Locks y Deadlock (4 condiciones Coffman)
5A	Sincronización — Semáforos
5B	Condvar (Variables de Condición)
5C	Monitores
6	Redes de Petri
7A	Pasaje de Mensajes / Channels
7B	Modelo de Actores
8	Sistemas Distribuidos (Sockets · Elección · Conocimiento)
9	Transacciones (ACID · 2PL · WAL · 2PC)
★	Problemas Clásicos (Pseudocódigo completo)
✓	Checklist + Verdadero/Falso típicos de examen

0. Rust Esencial para Concurrency

¿Por qué Rust?

- **Seguridad de memoria** en tiempo de compilación (sin GC): previene buffer overflow, dangling pointers, use-after-free.
- **Zero-cost abstractions**: abstracciones de alto nivel sin overhead en runtime.
- **Concurrency sin miedo**: el compilador previene condiciones de carrera en tiempo de compilación.

Ownership — Reglas fundamentales

- Cada valor tiene exactamente **un dueño (owner)** a la vez.
- Cambiar de dueño = hacer un **move**. El dueño original pierde el acceso.
- El dueño puede prestar: **múltiples referencias inmutables** (&T;) O **una única referencia mutable** (&mut; T), nunca ambas simultáneamente.
- Cuando el dueño sale de scope, el valor es **dropped** (trait Drop = RAII).

```
fn envejecer(mut p: Persona) { p.edad += 20; } // toma ownership → ariel no existe más
fn envejecer(p: &mut Persona) { p.edad += 20; } // presta ref mutable → ariel sigue vivo
```

Smart Pointers para concurrency

Tipo	Para qué	Thread-safe
Box<T>	Dato en el heap, único dueño	No aplica
Rc<T>	Múltiples referencias inmutables, mismo thread	No
Arc<T>	Múltiples referencias inmutables, múltiples threads (ref counting atómico)	Sí
Mutex<T>	Exclusión mutua para T (write lock)	Sí (con Arc)
RwLock<T>	Múltiples lectores O un escritor	Sí (con Arc)

Arc + RwLock — Patrón típico

```
use std::sync::{Arc, RwLock}; use std::thread; let data = Arc::new(RwLock::new(0i64)); let
data_clone = Arc::clone(&data); let handle = thread::spawn(move || { let mut w =
data_clone.write().unwrap(); // exclusive lock *w += 1; }); // lock liberado aquí por RAII (Drop)
handle.join().unwrap(); println!("{}", *data.read().unwrap()); // shared lock
```

Traits Send y Sync

- **Send**: el ownership del tipo puede transferirse entre threads. Casi todos los tipos son Send. Excepción: Rc<T>.
 - **Sync**: es seguro referenciar desde múltiples threads. T es Sync si &T; es Send. Excepción: Cell<T>, RefCell<T>.
 - Los tipos compuestos por tipos Send/Sync automáticamente implementan Send/Sync.
- Si intentás compartir algo que no es Send/Sync entre threads, el compilador te frena en tiempo de compilación.

Option y Result — manejo de errores

- `Option<T>`: `Some(T)` o `None`. Reemplaza `null`.
- `Result<T, E>`: `Ok(T)` o `Err(E)`. Para operaciones que pueden fallar.
- `.unwrap()` → `panic` si es `None/Err`. `.expect(msg)` → igual pero con mensaje. `?` → propaga el error.

1. Introducción — Definiciones y Modelos de Concurrency

Definiciones clave

Término	Definición
Programa	Instrucciones que se ejecutan secuencialmente en un procesador.
Proceso	Programa secuencial compuesto por instrucciones atómicas.
Programa concurrente	Conjunto finito de procesos secuenciales que pueden ejecutarse en paralelo. Ejecuta el intercalado arbitrario de instrucciones atómicas.
Instrucción atómica	Se ejecuta de principio a fin sin interrupciones, o no se ejecuta.
Paralelo	Las ejecuciones se superponen en tiempo en procesadores distintos.
Concurrente	Paralelismo potencial: las ejecuciones pueden superponerse.
Multitasking	Ejecución de múltiples procesos concurrentemente; el scheduler del SO coordina el acceso a los procesadores.
Multithreading	Feature del lenguaje: ejecución concurrente de más de un thread dentro de un programa.
Thread	Comparte recursos del proceso (espacio de memoria). Mantiene su propio stack, PC y registros.

Modelos de concurrency

- **Estado mutable compartido** — requiere serializar acceso (locks, semáforos, monitores).
- **Fork-Join** — tareas independientes en paralelo, resultados se unen.
- **Canales / Mensajes** — procesos se comunican enviando mensajes.
- **Programación asíncrona** — tareas livianas que ceden el control al esperar I/O.
- **Actores** — actores aislados que solo se comunican por mensajes asíncronos.

¿Cuándo usar cada modelo? — tabla examen

Caso de uso	Modelo	Justificación
Multiplicar matrices / Render 3D	Fork-Join	CPU-bound, tareas independientes, sin I/O
Llamadas a múltiples APIs / scraping	Async	I/O-bound, muchas esperas, alta latencia
Backend de chat / juego online	Actores o Async	Estado por conexión, reactividad, muchas conexiones
Procesamiento de logs masivos	Fork-Join o Async	Datos independientes → FJ; archivos → Async

Servidor que reenvía requests	Async	I/O pura, sin cómputo, máxima concurrencia
Sistema de subastas en tiempo real	Actores	Estado reactivo compartido, notificaciones
Crawler que descarga imágenes	Async	I/O-bound, muchas descargas simultáneas
Entrenamiento de red neuronal	Fork-Join	Cómputo intensivo, datos paralelizables
Problema de los filósofos / estado compartido	Threads + Mutex	Sincronización explícita, estado mutable compartido

■ Regla rápida: CPU-bound → Fork-Join | I/O-bound → Async | Estado reactivo → Actores

2. Modelo Fork-Join y Vectorización

Fork-Join

El cómputo (task) se parte recursivamente en sub-cómputos independientes (subtasks). Los resultados parciales se unen (join) para construir la solución final.

- Las tareas son **independientes** → se pueden ejecutar en paralelo.
- Tareas **no se bloquean** excepto para esperar el fin de sus subtareas.
- **Determinístico**: resultado siempre igual sin importar velocidad de threads.
- **Sin condiciones de carrera** porque los threads están aislados.
- **Performance ideal**: $t_{\text{secuencial}} / N_{\text{threads}}$.
- **Desventaja**: requiere que las unidades de trabajo estén aisladas.

Work Stealing

Algoritmo de scheduling. Worker threads inactivos roban trabajo a threads ocupados para balancear la carga.

- Cada thread tiene su propia **deque** (cola doble) con tareas listas.
- Al terminar tarea: agrega subtareas al **final** de su deque.
- Toma siguiente tarea del **final** de su deque (*LIFO propio*).
- Si deque vacía: **roba** del **inicio** de la deque de otro thread aleatorio (*FIFO ajeno*).
- Ventaja: baja sincronización, los workers solo se comunican cuando lo necesitan.

Rayon y Crossbeam en Rust

- **Rayon**: `rayon::join(f1, f2)` — dos tareas en paralelo. `.par_iter()` — itera en paralelo. Implementa work stealing internamente.
- **Crossbeam**: `crossbeam::scope` — garantiza que todos los threads terminan antes de salir del scope. Retorna Ok o Err (si alguno hizo panic).

```
// Rayon: map-reduce paralelo let result = data.par_iter().map(|x| process(x)).reduce(|| identity(), |a, b| merge(a, b));
```

Vectorización (SIMD)

Caso especial de paralelización **automática**. Al estancarse la Ley de Moore, se agregaron múltiples ALUs para operar sobre los mismos registros de forma concurrente → juegos de instrucciones SIMD (*Single Instruction, Multiple Data*): MMX, SSE, AVX (x86), NEON (ARM).

- Procesa una operación en múltiples pares de operandos **a la vez**.
- Cada variable (registro) es un **vector de tamaño fijo** (generalmente 128-512 bits).
- Se divide en N **lanes** (carriles): un registro de 512 bits puede alojar 16 enteros de 32 bits, o 8 flotantes de 64 bits.
- Casos de uso: sonido, imágenes, video, redes neuronales. Cada dato es independiente.

Operación	Descripción	Velocidad
Vertical	Entre distintos registros en el mismo lane. Ej: sumar vector A + vector B lane a lane.	Rápidas (paralelas)
Horizontal	En el mismo registro, entre distintos lanes. Reducen un vector a un escalar. Ej: $x_0 + x_1 + x_2 + x_3$.	Lentas (secuenciales)

3. Programación Asíncrona

¿Cuándo usar async?

Usar async	NO usar async (usar threads/sync)
Consultar servicios externos (HTTP, DB)	Calcular un factorial
Leer/escribir archivos	Multiplicar matrices
Servir múltiples requests concurrentes	Implementar filósofos (estado compartido)
Miles de conexiones I/O-bound	Cómputo CPU-intensivo puro

Futures

El trait `std::future::Future` representa una operación sobre la que se puede testear si se completó. El método `poll` **nunca bloquea**.

- Si completó → `Poll::Ready(output)`. Si no → `Poll::Pending`.
- Modelo **piñata**: lo único que se puede hacer con un future es hacer `poll` hasta que retorne el valor.
- Cada `poll` avanza todo lo que puede y almacena su estado para el siguiente `poll`.

```
trait Future { type Output; fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>; } enum Poll<T> { Ready(T), Pending }
```

Pin

Trait implementado por los Futures para garantizar que su referencia a sí mismos no sea movida en memoria una vez que se llamó a `poll()`. Evita self-references inválidas.

async fn y await

- Invocar una función `async` retorna **inmediatamente** un Future (sin ejecutar el cuerpo).
- Primer `poll`: ejecuta el cuerpo hasta el primer `await`.
- `await` toma ownership del future anidado y hace `poll`. Si `Pending` → retorna `Pending` al caller.
- El cambio de una tarea a otra ocurre **solo en los puntos await**.
- Un cómputo grande sin `await` no da lugar a otras tareas (diferencia clave con threads).

Executors

Función	Descripción
<code>block_on</code>	Función sincrónica que espera el resultado de una función <code>async</code> . Adaptador <code>sync↔async</code> . No usar dentro de código <code>async</code> (bloquea el thread).
<code>spawn_local</code>	Agrega future al pool que realiza polling en <code>block_on</code> . Lifetimes deben ser 'static. Todo en un único thread.
<code>task::spawn</code>	Crea la tarea en el pool de threads dedicado. No necesita <code>block_on</code> para ser polleada.
<code>yield_now</code>	Favorece el paralelismo: retorna <code>Pending</code> la primera vez, <code>Ready</code> la siguiente. Cede el control a otras tareas.

spawn_blocking	Para cálculos pesados/bloqueantes: coloca la tarea en otro thread del SO.
----------------	---

Concurrencia colaborativa: el thread duerme hasta que haya algo que pollear. La eficiencia viene de que el SO no necesita hacer context switch por interrupción, sino que el código mismo cede el control voluntariamente.

4. Corrección, Locks, Deadlock

Propiedades de corrección

Tipo	Propiedad	Descripción
Safety	Exclusión mutua	Dos procesos no deben intercalar instrucciones de su sección crítica.
Safety	Ausencia de deadlock	Un sistema no finalizado debe poder continuar avanzando productivamente.
Liveness	Ausencia de starvation	Todo proceso listo para usar un recurso debe recibirlo eventualmente.
Liveness	Fairness	Una instrucción continuamente habilitada eventualmente aparece en el escenario.

Safety: 'algo malo nunca ocurre'. Invariante que siempre debe mantenerse.

Liveness: 'algo bueno eventualmente ocurre'. El sistema eventualmente progresa.

Sección Crítica

Cada proceso ejecuta un loop infinito con parte crítica y parte no crítica.

- La sección crítica debe **progresar** (finalizar eventualmente).
- La sección no-crítica no requiere progreso.
- **3 especificaciones:** Exclusión mutua + Ausencia de deadlock + Ausencia de starvation.

4 Condiciones de Coffman para Deadlock

Para que haya deadlock deben cumplirse **las 4 simultáneamente**. Para prevenirlo: eliminar al menos UNA.

#	Condición	Descripción
1	Exclusión mutua	Al menos un recurso se retiene en modo exclusivo.
2	Espera y retención	Un proceso retiene recursos mientras espera otros.
3	No apropiación	Los recursos no se pueden quitar forzosamente a quien los tiene.
4	Espera circular	Existe una cadena circular: A espera a B, B espera a C, C espera a A.

Race Condition, Deadlock, Starvation, Busy Wait

Concepto	Definición	Solución
Race Condition	Resultado incorrecto por acceso concurrente no controlado a recursos compartidos.	Mutex, RwLock, mensajes
Deadlock	Bloqueo mutuo: A espera a B, B espera a A. Ninguno puede avanzar.	Orden global de locks, timeouts, Coffman

Starvation	Un thread nunca accede al recurso porque otros siempre lo toman antes.	Fairness en scheduler, prioridades
Busy Wait	Loop activo esperando una condición sin ceder la CPU. Consume CPU innecesariamente.	Condvar, semáforos, canales

Identificar Busy Wait: buscar un loop que verifica una condición sin blocking primitives (sin wait, sleep o recv).

Locks (RwLock y Mutex)

- **Mutex:** exclusión mutua total. `lock()` bloquea hasta adquirir. Retorna `MutexGuard<T>` que libera el lock con `RAII` (al salir de scope).
- **drop(guard):** libera el lock explícitamente antes de que salga de scope — importante para evitar deadlock en secciones largas.
- **Poison:** si un thread hace panic! con el lock tomado, el Mutex queda envenenado. Las llamadas a `lock()` retornan `Err`.
- **RwLock:** `read()` → lock compartido (varios lectores simultáneos). `write()` → lock exclusivo.
- **Lock envenenado RwLock:** ídem Mutex — panic con write lock envenena el RwLock.

```
use std::sync::{Arc, Mutex}; let data = Arc::new(Mutex::new(0i32)); let mut guard =
data.lock().unwrap(); *guard += 1; drop(guard); // libera ANTES de la siguiente línea (evita
deadlock) // ... código que no necesita el lock ... let guard2 = data.lock().unwrap();
```

5A. Semáforos

Definición y operaciones

Tipo de dato: entero no negativo V + conjunto de procesos L . Inicializa con $k \geq 0$ y $L = \emptyset$. **Si $V > 0$** : recurso disponible. **Si $V = 0$** : no disponible.

Operación	Efecto
<code>wait(S) / p(S)</code>	Si $V > 0$: $V \leftarrow V-1$. Si no: proceso bloqueado, se agrega a L .
<code>signal(S) / v(S)</code>	Si L vacío: $V \leftarrow V+1$. Si no: saca proceso q de L y lo pone ready.

- **Invariante:** $S.V = k + \# \text{signal}(S) - \# \text{wait}(S) \geq 0$.
- **Semáforo binario / Mutex:** $V \in \{0,1\}$. Equivale a un write lock.
- `wait()` y `signal()` son **instrucciones atómicas**.
- `signal()` despierta un proceso **arbitrario** (no determinístico quién).

Semáforos en Rust — `crate std::semaphore`

```
let sem = Semaphore::new(5); sem.acquire(); // wait - decrementa sem.release(); // signal - incrementa
let _guard = sem.access(); // wait con RAII (release automático al salir de scope)
```

Barreras — `std::sync::Barrier`

- `Barrier::new(n)` — sincroniza n threads en un punto.
- `barrier.wait()` — bloquea hasta que todos lleguen. Retorna `BarrierWaitResult`.
- `is_leader()` — true solo en el último thread que llama a `wait` (desbloquea al resto).
- Las barreras son **reutilizables** automáticamente.

5B. Variables de Condición (Condvar)

Definición

Mecanismo de sincronización que permite a un thread esperar hasta que cierta condición sea verdadera, **liberando el Mutex mientras espera**.

- No guarda ningún valor.
- Tiene asociado un FIFO de procesos esperando.
- Siempre se usa **junto con un Mutex** (patrón Arc<(Mutex, Condvar)>).

Operaciones (teóricas)

Operación	Efecto
waitC(cond)	Siempre bloquea al proceso. Libera el lock del monitor automáticamente. Agrega el proceso al FIFO.
signalC(cond)	Si hay procesos en el FIFO: saca el del tope y lo pone ready. Si no hay: no hace nada.
empty(cond)	Retorna true si no hay procesos esperando.

Condvar en Rust — std::sync::Condvar

```
use std::sync::{Arc, Mutex, Condvar}; let pair = Arc::new((Mutex::new(false), Condvar::new())); //
Waiter thread let (lock, cvar) = &*pair; let mut started = lock.lock().unwrap(); while !*started { //
<-- loop para manejar spurious wakeups started = cvar.wait(started).unwrap(); } // Notifier thread
let (lock, cvar) = &*pair; *lock.lock().unwrap() = true; cvar.notify_one(); // o notify_all()
```

■ **Siempre usar while, NO if, para verificar la condición. Los spurious wakeups (despertar sin que nadie haya notificado) son posibles en cualquier implementación de condvars.**

- wait_while(guard, |state| condicion) — forma idiomática en Rust: espera mientras la condición sea true.
- notify_one() — despierta un waiter arbitrario.
- notify_all() — despierta todos los waiters (útil cuando múltiples threads pueden avanzar).

Semáforo vs Monitor (Condvar)

Semáforo	Monitor / Condvar
wait puede o no bloquear	waitC siempre bloquea
signal siempre tiene efecto (V++)	signalC no tiene efecto si cola vacía
signal desbloquea proceso arbitrario	signalC desbloquea el tope de la cola FIFO
Proceso desbloqueado continúa inmediatamente	Proceso desbloqueado espera que el señalizador deje el monitor

5C. Monitores

Definición

Herramienta de sincronización que provee exclusión mutua implícita y condvars. Solo un proceso puede ejecutar el monitor a la vez.

Componentes

- Nombre, variables internas, procedimientos (acceden a variables internas), interfaz pública, inicialización, condition variables.

Estados de proceso en el monitor

- Esperando para entrar al monitor.
- Ejecutando el monitor (solo un proceso — exclusión mutua).
- Bloqueado en FIFO de condition variable.
- Recién liberado de waitC.
- Recién completó una operación signalC.

Monitores en Java — synchronized

- **Bloque synchronized:** `synchronized(obj) {{ ... }}` — el objeto es el lock (monitor).
- **Método synchronized:** el lock es `this` (o la clase para `static`).
- El lock es **reentrante**: un thread puede re-entrar en el mismo bloque `synchronized`.
- `wait()` — libera el monitor y suspende el hilo hasta `notify/notifyAll`.
- `notify()` — despierta un hilo arbitrario que espera por el monitor.
- `notifyAll()` — despierta todos los hilos que esperan.
- `volatile` — la variable siempre se lee de memoria principal (no cachea). No hace locking.

6. Redes de Petri

Definiciones

Herramienta matemática y visual para modelar concurrencia y sincronización.

Red	Definición formal
Ordinaria	$PN = (T, P, A)$ — T : transiciones, P : lugares, $A \subseteq (T \times P) \cup (P \times T)$: arcos
General	$PN = (T, P, A, W, M_0)$ — agrega $W: A \rightarrow \mathbb{N}$ (peso arcos), $M_0: P \rightarrow \mathbb{N} \cup \{0\}$ (marca inicial)

Reglas de disparo

- Transición t **habilitada** $\iff M(p) \geq W(p,t)$ para todo $p \in I(t)$.
- Al disparar: $\forall p \in I(t): M(p) \leftarrow M(p) - W(p,t)$ (consume tokens de entradas)
- $\forall p' \in O(t): M(p') \leftarrow M(p') + W(p',t)$ (produce tokens en salidas)
- **Función de marca:** $M: P \rightarrow \mathbb{N} \cup \{0\}$ — tokens en cada lugar.
- $I(t)$ = lugares de entrada de t . $O(t)$ = lugares de salida de t .

Lugares entrada	Transiciones	Lugares salida
Precondiciones	Eventos	Postcondiciones
Datos de entrada	Cómputos	Datos de salida
Buffers de entrada	Procesador	Buffers de salida

Arco inhibidor

Extensión de la Red General. Un arco inhibidor de P a T hace que la transición T esté habilitada solo si $M(p) = 0$ (el lugar está vacío). Se representa con un círculo en lugar de flecha. Permite modelar **ausencia de tokens**.

- Caso de uso: Lector-Escritor — la transición 'comenzar a leer' está inhibida si hay un escritor.

7A. Pasaje de Mensajes y Canales

Modelos de comunicación

Dimensión	Opciones
Comunicación	Sincrónica (emisor espera al receptor) / Asíncrona con buffer
Direccionamiento	Simétrico (ambos se conocen) / Asimétrico / Sin direccionamiento (por estructura del mensaje)
Flujo de datos	Unidireccional / Bidireccional

Canales en Rust — `std::sync::mpsc`

■ "Do not communicate by sharing memory; instead, share memory by communicating."

- `mpsc` = **M**ultiple **P**roducer, **S**ingle **C**onsumer.
- Transfieren **ownership** del elemento enviado (garantía del borrow checker).
- Para múltiples productores: clonar el transmisor `tx.clone()`.

```
use std::sync::mpsc; let (tx, rx) = mpsc::channel(); // sin buffer (sincrónico) let (tx, rx) = mpsc::sync_channel(10); // con buffer de 10 (asincrónico hasta llenarse) tx.send(valor).unwrap(); // transfiere ownership rx.recv().unwrap(); // bloquea hasta recibir rx.try_recv(); // no bloquea, retorna Result
```

Selective Input

Sintaxis para escuchar en varios canales de forma bloqueante, desbloqueándose con el primero que recibe mensaje.

```
either ch1 => var1 or ch2 => var2
```

Canales en Unix

- **Pipes:** orientados a bytes. Sin representación en filesystem.
- **FIFOs:** como pipes, con representación en filesystem (nombre).
- **Message Queues:** mensajes como unidades independientes.

7B. Modelo de Actores

Propiedades del actor

Desarrollado por Carl Hewitt (1973), popularizado por Erlang. Similar al pasaje de mensajes.

- **Livianos:** se pueden crear miles en lugar de threads.
- Encapsulan **comportamiento y estado** (privado).
- **Componentes:** dirección (mailbox address) + mailbox (FIFO de mensajes).
- **Aislados:** no comparten memoria. Estado privado solo cambia procesando mensajes.
- Procesan **un mensaje por vez**.
- En sistemas distribuidos, la dirección puede ser remota.
- El actor **supervisor** puede crear actores hijo.

Ciclo de vida (Actix)

Estado	Descripción
Started	Método started(). Contexto disponible.
Running	Estado normal. Puede durar indefinidamente.
Stopping	Al llamar Context::stop, sin referencias externas, o sin objetos en contexto.
Stopped	Estado final.

Formas de enviar mensajes (Actix)

Método	Comportamiento
do_send(M)	Ignora errores. Si mailbox cerrado, descarta. No retorna resultado.
try_send(M)	Inmediato. Si mailbox lleno o cerrado, retorna SendError.
send(M)	Retorna Future con el resultado del handler del mensaje.

Filósofos con Actores — Solución Chandy/Misra

Solución distribuida al problema de los filósofos usando pasaje de mensajes (sin estado compartido).

- Para cada par de filósofos que compiten por un palito, se crea un palito asignado inicialmente al de **ID más bajo** (evita deadlock).
- Cada palito puede estar **sucio** o **limpio**. Inicialmente todos sucios.
- Para comer, un filósofo pide los palitos que no tiene a sus vecinos.
- Si tiene el palito y está **limpio**: lo retiene (vecino queda en espera).
- Si tiene el palito y está **sucio**: lo limpia y lo entrega.
- Al terminar de comer: todos sus palitos quedan **sucios**. Si le habían solicitado alguno, lo limpia y envía.

✓ Esta solución garantiza ausencia de deadlock y starvation.

8. Sistemas Distribuidos

Entidades y capacidades

Una entidad es la unidad de cómputo en un ambiente distribuido (proceso, hilo, nodo, etc.).

- **Capacidades:** lectura/escritura en memoria local, procesamiento local, comunicación (preparar/enviar/recibir mensajes), reloj local.

Evento	Código	Descripción
Llegada de mensaje	RECEIVE	La entidad recibe un mensaje de otra.
Alarma del reloj	ALARM	Se cumplió un tiempo programado.
Evento espontáneo	IMPULSE	Ocurre sin causa externa (ej: inicio del sistema).

Restricciones temporales

- **Delays acotados:** cualquier mensaje tarda como máximo Δ .
- **Delays unitarios:** todos los mensajes tardan exactamente 1 unidad.
- **Relojes sincronizados:** todos los relojes avanzan igual.

Costo y complejidad en distribuidos

- El **cómputo local es gratis**; el costo está en la **comunicación**.
- **Costo de mensajes:** número total de mensajes enviados.
- **Complejidad temporal:** número de rounds (pasos de comunicación).

Sockets

Punto final de comunicación bidireccional entre dos procesos. **Diferencia con channels:** channels son para threads del mismo proceso (en memoria, seguros por tipo). Sockets son para procesos distintos, incluso en máquinas diferentes.

Tipo	Protocolo	Características
Stream	TCP	Orientado a conexión. Entrega garantizada y ordenada.
Datagram	UDP	Sin conexión. Rápido. Sin garantías de entrega.
Unix Domain	Local	Comunicación local (misma máquina). No usa red.

Deadlocks en sistemas distribuidos

Más difícil que en sistemas centralizados: no hay visión global del estado ni memoria compartida entre nodos.

Algoritmo Centralizado

- Un **coordinador** mantiene el grafo de uso de recursos (**Wait-For Graph**).
- Los procesos envían mensajes al coordinador cuando obtienen o liberan recursos.
- **Problema:** mensajes pueden llegar desordenados → falsos deadlocks.
- **Solución:** timestamps globales (Lamport) para ordenar los mensajes.

Algoritmo Distribuido (*Probe Messages*)

- Cuando un proceso P espera un recurso de Q, envía un **probe message** a Q.

- El mensaje contiene: (id_bloqueado, id_sender, id_destinatario).
- Q reenvía el probe a todos los procesos de los que espera recursos.
- Si el mensaje llega de vuelta a P → hay un ciclo → **deadlock detectado**.

Acciones al detectar deadlock: abortar un proceso involucrado, liberar recursos de procesos no críticos, elegir víctima según: tiempo de ejecución, prioridad, recursos usados.

Algoritmos de Elección de Líder

Bully Algorithm

- Proceso P nota que el coordinador no responde e inicia la elección.
- P envía ELECTION a todos los procesos con **ID mayor**.
- Si nadie responde → P gana y anuncia COORDINATOR.
- Si alguien con ID mayor responde OK → ese proceso continúa la elección.
- Siempre gana el proceso con **mayor ID**.

Ring Algorithm

- Los procesos están en un **anillo lógico**.
- Al detectar falla del coordinador, se envía ELECTION con el propio ID al sucesor.
- Cada proceso agrega su ID a la lista y la reenvía.
- Cuando el mensaje vuelve al iniciador, tiene todos los IDs → elige el **mayor**.
- Envía COORDINATOR con el nuevo líder a todos.

Conocimiento distribuido

- **Conocimiento local**: lo que la entidad sabe por sí misma.
- **Conocimiento distribuido**: lo que el sistema como conjunto sabe (aunque ninguna entidad individual lo sepa todo).
- **Conocimiento común**: todos saben que todos saben. Muy difícil de lograr en sistemas distribuidos.

9. Transacciones y Persistencia

Propiedades ACID

Letra	Propiedad	Significado
A	Atomicidad	Se ejecuta completa o no se ejecuta. No hay estados intermedios visibles.
C	Consistencia	Lleva el sistema de un estado válido a otro estado válido.
I	Aislamiento	Las transacciones concurrentes no se interfieren entre sí.
D	Durabilidad	Una vez commitada, la transacción persiste aunque el sistema falle.

Control de Concurrencia

2-Phase Locking (2PL)

Protocolo que garantiza **serialización** de transacciones.

- **Fase de expansión:** la transacción solo adquiere locks, no libera ninguno.
- **Fase de contracción:** la transacción solo libera locks, no adquiere nuevos.
- Garantiza serialización pero puede causar deadlocks.
- **Mejor para:** alta contención, cuando la corrección es prioritaria.

Timestamps

- Cada transacción recibe un timestamp al inicio. Se decide si puede leer/escribir comparando timestamps.
- No requiere locks → evita deadlocks. Problema: muchos abortos en alta contención.
- **Mejor para:** baja contención.

Concurrencia Optimista (OCC)

- Las transacciones ejecutan sin bloquear nada. Al hacer commit se valida que no hubo conflictos.
- Fases: lectura → validación → escritura.
- Si hay conflicto en la validación: aborta y reintenta.
- **Mejor para:** baja contención, lectura dominante.

Mecanismo	Deadlocks	Abortos	Mejor para
2PL	Posibles	Pocos	Alta contención
Timestamps	No	Frecuentes	Baja contención
OCC	No	Frecuentes	Baja contención, read-heavy

WAL vs Private Workspace

	Write-Ahead Log (WAL)	Private Workspace
--	-----------------------	-------------------

Mecanismo	Modifica in-place pero primero escribe en el log. Al commit: escribe COMMIT en log. Al abortar: rollback leyendo log hacia atrás.	Al iniciar, el proceso recibe copia de los archivos. Trabaja sobre la copia. Al commit: persiste la copia al archivo real.
Ventaja	Durabilidad, recuperación ante fallas.	Aislamiento natural, menos overhead de logging.
Desventaja	Cada escritura se duplica (log + archivo).	Copia inicial costosa si los archivos son grandes.
Uso	Preferido en sistemas distribuidos por robustez ante fallas.	Mejor en sistemas con archivos chicos y pocas transacciones.

2-Phase Commit (2PC) — transacciones distribuidas

Protocolo para garantizar que una transacción distribuida se commitea o aborta en **todos** los nodos.

- **Fase 1 — Preparación:** el coordinador pregunta a todos '¿podés commitear?'. Cada participante responde Sí (y persiste en WAL) o NO.
- **Fase 2 — Decisión:** si todos Sí → coordinador envía COMMIT. Si alguno NO → envía ABORT.
- **Ventaja:** garantiza atomicidad distribuida.
- **Desventaja:** si el coordinador falla entre fases, los participantes quedan **bloqueados**.

★ Problemas Clásicos — Pseudocódigo completo

Problema 1: Productor-Consumidor

Productores generan ítems; consumidores los consumen de un buffer compartido.

- Premisas: no consumir lo que no hay; todos los ítems producidos son consumidos; acceso al buffer de a uno; respetar orden FIFO.

Buffer Infinito — con Semáforos

```
sem notEmpty(0) Productor: Consumidor: loop forever loop forever producir d wait(notEmpty) append(d, buffer) d <- take(buffer) signal(notEmpty) consumir(d)
```

Buffer Acotado (tamaño N) — con Semáforos

```
sem notEmpty(0), notFull(N) Productor: Consumidor: loop forever loop forever producir d wait(notEmpty) wait(notFull) d <- take(buffer) append(d, buffer) signal(notFull) signal(notEmpty) consumir(d)
```

Buffer Acotado — con Condvar (Rust)

```
let pair = Arc::new((Mutex::new(VecDeque::new()), Condvar::new())); // Productor: let mut buf = cvar_pair.0.lock().unwrap(); buf = cvar_pair.1.wait_while(buf, |b| b.len() >= N).unwrap(); buf.push_back(item); cvar_pair.1.notify_all(); // Consumidor: let mut buf = cvar_pair.0.lock().unwrap(); buf = cvar_pair.1.wait_while(buf, |b| b.is_empty()).unwrap(); let item = buf.pop_front().unwrap(); cvar_pair.1.notify_all();
```

Problema 2: Barbero Durmiente

Una barbería tiene sala de espera con sillas. Si está vacía, el barbero duerme. Si entra un cliente y el barbero duerme, lo despierta. Si está atendiendo, el cliente espera.

Con Semáforos

```
sem customer_waiting(0) // clientes esperando al barbero sem barber_ready(0) // barbero listo para atender sem haircut_done(0) // corte terminado Barbero: Cliente: loop forever loop forever wait(customer_waiting) signal(customer_waiting) // avisa que llegó signal(barber_ready) wait(barber_ready) // espera al barbero cortar_pelo() sentarse_en_silla() signal(haircut_done) wait(haircut_done) // espera fin de corte
```

Con Condvar (Rust — estructura)

```
// Arc<(Mutex<State>, Condvar)> // State { waiting_customers: usize, barber_sleeping: bool } // Barbero: let mut state = lock.lock().unwrap(); if state.waiting_customers == 0 { state.barber_sleeping = true; } state = cvar.wait_while(state, |s| s.waiting_customers == 0).unwrap(); state.waiting_customers -= 1; state.barber_sleeping = false; // ... cortar pelo ... // Cliente: let mut state = lock.lock().unwrap(); state.waiting_customers += 1; if state.barber_sleeping { cvar.notify_one(); } // esperar fin de corte...
```

Problema 3: Fumadores

3 ingredientes: tabaco, papel, fósforos. 3 fumadores, cada uno con un ingrediente en cantidad infinita. Un agente pone aleatoriamente 2 ingredientes en la mesa. El fumador que tiene el tercero toma los 2 y fuma.

Con Semáforos

```
// Un semáforo por fumador (ingredient_sem[0,1,2]) // sem table_free(1) para que el agente espere Agente: Fumador i: loop forever loop forever wait(table_free) wait(ingredient_sem[i]) ings = random 2 of {0,1,2} fumar() for ing in ings: signal(table_free) signal(ingredient_sem[ing])
```

Con Condvar (Rust — estructura)

```
// Arc<(Mutex<[bool;3]>, Condvar)> // ings[i] = hay ingrediente i en la mesa // Agente: let mut ings = lock.lock().unwrap(); cvar.wait_while(ings, |i| i.iter().any(|x| *x)).unwrap(); // espera mesa libre ings[a] = true; ings[b] = true; cvar.notify_all(); // Fumador i (tiene ingrediente i, necesita los otros dos): let mut ings = lock.lock().unwrap(); ings = cvar.wait_while(ings, |i| !(i[j] &&
```

```
i[k])).unwrap()); // espera sus 2 ins[j] = false; ins[k] = false; cvar.notify_all(); // avisa que la mesa quedó libre
```

Problema 4: Filósofos Comensales

5 filósofos, 5 palitos. Para comer necesita el palito de su izquierda y el de su derecha. Si todos agarran el izquierdo simultáneamente → deadlock.

Solución: Romper simetría (el N-ésimo toma al revés)

```
// Filósofo i: if i == N-1: first = chopstick[next] // N-ésimo agarra DERECHA primero second = chopstick[i] else: first = chopstick[i] // los demás agarran IZQUIERDA primero second = chopstick[next] loop: pensar() acquire(first) // semáforo binario por palito acquire(second) comer() release(first) release(second)
```

✓ Romper la simetría garantiza ausencia de deadlock. Un filósofo siempre podrá comer.

Solución: Mozo mediador (channel/actor)

- Un mozo controla el acceso a los palitos. Los filósofos le piden al mozo, no directamente entre sí.
- El mozo puede implementar políticas de fairness (ej: no dar un palito si el filósofo adyacente lleva mucho tiempo esperando).

Problema 5: Lector-Escritor

Un estado compartido. Múltiples procesos leen (sin conflicto entre sí) o escriben (exclusivo). Mientras un escritor escribe, nadie puede leer ni escribir.

Normal (sin preferencia) — con Condvar

```
// Arc<(Mutex<State>, Condvar)> // State { readers: u32, writing: bool } // Lector: let mut s = cvar.wait_while(lock.lock().unwrap(), |s| s.writing).unwrap(); s.readers += 1; drop(s); // ... leer ... lock.lock().unwrap().readers -= 1; cvar.notify_all(); // Escritor: let mut s = cvar.wait_while(lock.lock().unwrap(), |s| s.writing || s.readers > 0).unwrap(); s.writing = true; drop(s); // ... escribir ... lock.lock().unwrap().writing = false; cvar.notify_all();
```

Con preferencia de escritura (writer preference)

Agregar campo writers: u32 al estado. Un lector espera si writing || writers > 0. El escritor incrementa writers antes de esperar la sección crítica, garantizando que nuevos lectores no entren mientras hay escritores esperando.

✓ Writer preference resuelve starvation de escritores pero puede causar starvation de lectores.

✓ Checklist y Verdadero/Falso típicos de Examen

Verdadero/Falso — los más frecuentes

Afirmación	R	Justificación
El estado interno de un actor está protegido por exclusión mutua.	F	Es privado: solo el actor accede. No necesita mutex. La serialización la da el mailbox.
La llamada al método poll de un Future es bloqueante.	F	poll() nunca bloquea. Retorna Ready o Pending inmediatamente.
Programando en fork-join hay que cuidar las race conditions.	F	Las subtareas no comparten estado mutable; no hay races.
Un hilo en Condvar solo despierta cuando otro hace signal.	F	Existen spurious wakeups. Siempre usar while, no if.
Las tareas asíncronas son ideales para cómputo intensivo.	F	Async es para I/O-bound. El cómputo intensivo usa fork-join.
El estado de un actor puede accederse con locks.	F	Solo mediante mensajes. No hay acceso directo desde afuera.
La función async inicia la operación cuando se la llama.	F	Retorna inmediatamente un Future. Inicia recién cuando se hace poll (await).
En un entorno de 1 CPU, threads intensivos son peores que async.	V	Async tiene menos overhead; threads compiten por la única CPU.
En Fork-Join, las tareas se pueden bloquear libremente.	F	Solo se bloquean para esperar el fin de sus subtareas.
WAL garantiza durabilidad ante fallas de sistema.	V	Escribe en log antes que en archivo. Permite rollback y recovery.
Dos transacciones pueden ejecutarse en paralelo sin problema.	F	Pueden generar inconsistencias sin control de concurrencia (2PL/OCC/timestamps).
drop(guard) en Rust libera un lock antes de que salga de scope.	V	Necesario para evitar deadlock cuando el scope es largo.

Regla rápida para elegir modelo

- **CPU-bound / cómputo / datos paralelizables** → Fork-Join
- **I/O-bound / red / muchas conexiones / esperas** → Async
- **Estado reactivo / notificaciones / sistema de mensajes** → Actores
- **Siempre justificar** por qué el otro modelo no sirve.

Estructura de respuesta: diseño con Actores

- 1. **Identificar entidades** del sistema → cada entidad = un actor.
- 2. Para cada actor: **estado interno** (struct en Rust).
- 3. Para cada actor: **mensajes que recibe** (structs + impl Message).

- 4. Para cada actor: **handler** — qué hace al recibir cada mensaje (impl Handler).
- 5. **Interacciones**: quién envía qué a quién. `do_send` (fire&forget;) vs `send` (retorna Future).

Conceptos que tenés que saber definir

- Instrucción atómica, proceso, programa concurrente.
- Safety vs Liveness. Ejemplos de cada una.
- Exclusión mutua, deadlock, starvation, fairness, busy wait.
- Sección crítica: las 3 especificaciones.
- 4 condiciones de Coffman para deadlock.
- Semáforo: definición formal, invariante, wait/signal.
- Diferencias semáforo vs condvar (tabla de 4 filas).
- Monitor: componentes, exclusión mutua implícita.
- Future: qué es poll, Pending vs Ready. Modelo piñata.
- Fork-Join: determinismo, work stealing.
- Actor: mailbox, aislamiento, `do_send` vs `try_send` vs `send`.
- Red de Petri: función de marca, reglas de disparo.
- ACID: qué significa cada letra.
- 2PL: las dos fases. 2PC: las dos fases.
- WAL vs Private Workspace.
- Bully vs Ring: cómo gana el líder en cada uno.

Código que tenés que poder escribir

- `Arc>` — leer/escribir con guards RAII. `drop(guard)` explícito.
- `Arc>` — `lock()` + `drop()` para liberar antes de tiempo.
- `Arc<(Mutex, Condvar)>` — `wait_while` / `notify_one` / `notify_all`.
- Implementación de Semáforo con Condvar (wait/signal).
- Productor-Consumidor: buffer infinito y acotado con semáforos.
- Barbero durmiente con semáforos.
- Fumadores con semáforos o condvar.
- Filósofos: romper simetría (N-ésimo al revés).
- `thread::spawn` + `join`, Rayon `par_iter`, Crossbeam `scope`.
- `mpsc::channel` — `send` / `recv` / `try_recv`.
- `async fn` + `await`, `block_on`.
- Actor en Actix: `trait Actor`, `impl Handler`, `addr.do_send()`.

Errores comunes a evitar

- Usar `if` en lugar de `while` al esperar en condvar (spurious wakeups).
- Olvidar que `signal()` del semáforo siempre tiene efecto (V++), distinto de `signalC()`.
- Confundir `Rc` (no thread-safe) con `Arc` (thread-safe).
- No hacer `drop()` del guard antes de notificar → puede causar deadlock.
- En Fork-Join: las tareas solo se bloquean para esperar subtareas.
- Filósofos: si todos agarran primero izquierda → deadlock. Romper simetría.
- `poll()` NUNCA bloquea. Si decís que bloquea en un V/F → FALSO.
- El actor accede a su estado SOLO procesando mensajes, no con locks.

■ **2PC: si el coordinador falla entre fases → participantes bloqueados indefinidamente.**

Resumen v3 — Técnicas de Programación Concurrente I · FIUBA · Basado en clases Deymonnaz + prácticas + resúmenes de compañeros